

THREADS

Temporal Historical Relational Engine for Adversarial Deterministic Systems

Contact: rickeyernest1990@gmail.com

Publication date: 6 September 2026

Contents

Overview	1
1 Motivation and scope	1
2 Architecture	2
2.1 Event model	2
2.2 Execution flow	2
3 Temporal Ledger mathematics	3
4 Relational propagation	4
4.1 Semirings	4
4.2 Exact functional alternatives	5
4.3 Contradiction, negation and proof	5
5 Finite joins and bounded program search	5
6 Experimental protocol and fairness	6
7 Ten scientific headline experiments	6
7.1 Selected interpretation	7
8 Ten applied AI and enterprise experiments	7
9 Five diagnostic microtests	8
10 What the experiments currently support	9
11 What the experiments do not support	9
12 Minimal reference use and reproduction	10

Licensing warning - noncommercial use. This publication uses a split licence. The documentation, explanatory text, figures, tables, and other copyrightable presentation in this PDF are licensed under Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0). Source code and code listings implementing THREADS are licensed separately under the PolyForm Noncommercial License 1.0.0. Please review the applicable licence before copying, adapting, distributing, or reusing material from this publication.

Overview

THREADS is an experimental deterministic engine for structured temporal state and relational reasoning. It stores versioned events, keeps positive and negative evidence separate, answers current or historical queries, propagates finite relation programs, preserves unresolved alternatives, and records event provenance.

Origin. THREADS grew out of an unsuccessful attempt to build a new transformerless AI architecture. That larger attempt failed as a complete AI system. During the work, however, one deterministic section—the temporal Ledger and relational reasoning code—kept performing well in smaller controlled tests. I therefore separated and repurposed that code as a standalone research project. THREADS is the result. The tested Python package still uses the earlier internal name `srmh` so the source hashes and benchmark history remain reproducible.

Experimental status. The results are controlled synthetic tests, not peer-reviewed evidence of superiority over Transformers, large language models, databases, SAT/SMT solvers, or production rule engines. They support narrower claims about the tested implementation: exact finite relational propagation, temporal replay and revision, explicit negation and contradiction, exact handling of reconverging alternatives, finite joins, provenance, and bounded operator-program search.

1 Motivation and scope

The research question is deliberately narrow:

Can a deterministic engine maintain changing relational state, reason over long structured chains, preserve history and contradictions, isolate irrelevant records, and return an auditable answer without being given the expected result or an oracle-selected path?

THREADS is not a language model. The Ledger/Engine path has no trainable parameters. The inputs used here are already structured as exact identifiers, relations, events, tables, or operator demonstrations. Ordinary relation walks are given the relation program to execute; only the bounded program-search experiment infers an operator sequence from examples.

2 Architecture

2.1 Event model

The source code stores a Ledger event as

$$e = (i, s, r, o, t, \omega, \sigma, c, \gamma, \pi, \ell, \tau), \quad (1)$$

where i is the event ID, s subject, r relation, o object, t event time, $\omega \in \{\text{ASSERT}, \text{RETRACT}\}$, σ source, c context, $\gamma \in [0, 1]$ confidence metadata, $\pi \in \{+, -\}$ polarity, ℓ optional lineage, and τ an optional retraction target.

Event IDs and the main relational fields are exact strings. Querying does not modify the Ledger. Confidence is used as edge support only by MAX_MIN; it is not used to choose a winner between competing functional values in the tested symbolic engine.

2.2 Execution flow

Figure 1 shows the information flow in the tested implementation. The Engine reads active Ledger state, applies a selected semiring during the walk, and returns a final status, values, and optional provenance. Natural join and bounded program search are separate exact utilities rather than a compulsory stage in every walk.



Figure 1: THREADS information flow. Natural-language parsing is outside the tested core.

3 Temporal Ledger mathematics

For one subject, relation and context, let

$$E_\theta = \{e : t_e \leq \theta\} \quad (2)$$

be the events eligible at query time θ .

For a functional relation, the implementation first finds the latest *positive assertion timestamp*

$$t^+ = \max(\{t_e : e \in E_\theta, \omega_e = \text{ASSERT}, \pi_e = +\} \cup \{-\infty\}). \quad (3)$$

Positive assertions from earlier timestamps are superseded; negative assertions are retained independently. A candidate assertion e is closed if an eligible retraction with matching object

and polarity occurs at or after e , and that retraction is either untargeted or targets e directly. Thus

$$A_\theta = \{e \in E_\theta : \omega_e = \text{ASSERT}, e \text{ survives functional selection and is not closed}\}. \quad (4)$$

This ordering matters: functional selection is based on assertion time before retraction filtering. Therefore retracting the newest functional value does *not* automatically revive an older superseded value in the tested policy.

Let

$$P = \{o_e : e \in A_\theta, \pi_e = +\}, \quad N = \{o_e : e \in A_\theta, \pi_e = -\}. \quad (5)$$

The direct Ledger lookup reports

$$\text{status} = \begin{cases} \text{CONTRADICTED}, & P \cap N \neq \emptyset, \\ \text{AMBIGUOUS}, & r \text{ functional and } |P| > 1, \\ \text{ANSWER}, & P \neq \emptyset, \\ \text{NEGATED}, & P = \emptyset, N \neq \emptyset, \\ \text{UNKNOWN}, & \text{otherwise.} \end{cases} \quad (6)$$

This is a local cell status. The Engine does not permanently latch an intermediate **AMBIGUOUS** or **CONTRADICTED** label; it carries the relevant alternatives through the remaining relation program and decides the final status from the terminal outcomes.

4 Relational propagation

4.1 Semirings

For a finite relation program $(r_1, \dots, r_d)$ and semiring

$$\mathcal{S} = (\oplus, \otimes, 0, 1), \quad (7)$$

with initial frontier $F_0(s_0) = 1$, propagation has the standard form

$$F_j(v) = \bigoplus_u F_{j-1}(u) \otimes w(u, r_j, v). \quad (8)$$

Algebra	$\oplus$	$\otimes$	0	1
MAX_MIN	max	min	0	1
COUNT	+	$\times$	0	1
MIN_PLUS	min	+	∞	0
BOOLEAN	$\vee$	$\wedge$	false	true

Table 1: The four propagation algebras in `algebra.py`.

The standalone **Graph** class uses its stored edge weight w . The Ledger-backed Engine maps events differently: **MAX_MIN** uses the maximum active confidence for each positive object; **COUNT** uses unit edges; **MIN_PLUS** uses unit cost and therefore measures hop distance; **BOOLEAN** uses a true/unit edge. Multiple active positive events for the same subject–relation–object are collapsed to one object edge in the Engine, so duplicate event records do not multiply **COUNT**.

4.2 Exact functional alternatives

Suppose a functional cell $k = (c, s, r)$ has possible objects in a finite domain D_k . A contribution carries a symbolic condition ϕ , such as $(k = a)$.

When every value in a complete choice domain produces the same payload under the same remaining conditions, the choice can be removed exactly:

$$\{\psi \wedge (k = v) \mapsto z : v \in D_k\} \implies \{\psi \mapsto z\}. \quad (9)$$

This is not fuzzy pruning. It removes only a distinction that can no longer change the current payload.

At the terminal step, let K be the choice keys that still matter and let each exact assignment $a \in \prod_{k \in K} D_k$ produce status S_a and frontier F_a . Define

$$\Omega = \{(S_a, F_a) : a \in \prod_{k \in K} D_k\}. \quad (10)$$

If $|\Omega| = 1$, that outcome is returned. If several distinct outcomes remain, the query is **AMBIGUOUS**. Mutually exclusive numeric frontiers are kept separate rather than merged into a number that occurs in no possible outcome.

4.3 Contradiction, negation and proof

If a traversed positive edge points to an object that also has active negative evidence, that path becomes contradiction-tainted. Taint accumulates along a path, but when compatible derivations reach the same endpoint under the same condition, one clean derivation is enough to keep that endpoint clean. Explicit terminal negative evidence remains **NEGATED**; absence without negative evidence remains **UNKNOWN**.

Proof edges store the supporting event IDs and the functional-choice condition under which each edge exists. The witness extractor rejects any proof path that would require incompatible values of the same functional cell. Definite answers can therefore expose one compact world-consistent witness while retaining the larger proof DAG internally.

5 Finite joins and bounded program search

For binding rows a and b , natural join is

$$R \bowtie S = \{a \cup b : a \in R, b \in S, a \text{ and } b \text{ agree on shared variables}\}. \quad (11)$$

The implementation changes only execution order: it starts with the smallest table and then prefers constraints that overlap variables already bound. This reduces avoidable intermediate products without changing the final relation.

Bounded program search receives a supplied finite operator library $\mathcal{O}$ and demonstrations (x_i, y_i) . It keeps programs $p \in \mathcal{O}^{\leq d}$ satisfying

$$\sum_i \mathbf{1}[p(x_i) \neq y_i] \leq \varepsilon. \quad (12)$$

Programs with identical demonstration behavior share execution work, but alternative programs remain represented. Held-out prediction is **AMBIGUOUS** when retained programs disagree and **UNKNOWN** when an incomplete search cannot certify an answer.

6 Experimental protocol and fairness

The benchmark rules were fixed to keep the engine separate from the grader.

- **No answer leakage.** THREADS receives the structured problem and query. Expected answers and independent oracles are used only after execution for scoring.
- **No oracle path selection.** Distractors stay in the same Ledger/graph. The useful path is not extracted into a smaller input before the query.
- **Unsorted temporal ingestion.** Temporal and several applied tests deliberately shuffle event insertion order. THREADS reconstructs state from event time.
- **Raw with respect to ordering/relevance, not language.** Records are not pre-labelled as relevant/irrelevant or oracle-sorted into an answer path before ingestion, but they are already structured subject/relation/object/time records rather than raw natural language.
- **Supplied programs.** Ordinary walks receive the relation sequence to execute. Only the bounded program-search benchmark infers a short operator sequence from demonstrations.

The principal stored paper run used Python 3.13.5 on Linux with five vCPUs exposed to the container. Timings are single-run observations, not hardware-normalized averages. Source integrity is recorded by SHA-256; the three main files hash to:

```
engine.py  91ba71a0c128a813...d6096f8c5379ce0701b49ebf9cda6481
ledger.py  ce08fca2934988b1...5bd06d584a32de029f0cf0a689d3801e
algebra.py f5b759597be4069e...810a68adbb9bd677e0b3bf8cddf9f29
```

7 Ten scientific headline experiments

All ten predefined headline experiments passed on the stored snapshot.

#	Experiment	Scale	Stored result
1	Semiring differential correctness	10,000 random layered graphs	10,000/10,000 matched independent walk enumeration; 2,500 per algebra
2	Long-depth reasoning	1k–200k hops	Exact at every depth; 200k query 0.645 s
3	Indexed distractor scaling	128 useful hops + up to 1M distractors	Exact at all scales; 1M query 0.000543 s; process peak about 1.0 GB RSS
4	Temporal replay	50,000 shuffled events; 5,000 historical oracle queries	0 mismatches; 1,000 Engine transport checks also passed
5	Revision / dependency invalidation	1,000 depth-20 histories	2,000/2,000 historical/current checks; 400 unchanged, 300 retracted, 300 superseded

Table 2: Scientific headline results, part I.

#	Experiment	Scale	Stored result
6	Possible outcomes / reconvergence	40,000 random cases + 1,000 binary choices	40,000/40,000 exact; reconvergence kept max frontier 2 and one final outcome
7	Renaming and insertion-order invariance	1,000 transformed problems	1,000/1,000 invariant
8	Proof/provenance integrity	2,000 proof cases	2,000/2,000 event-backed, choice-consistent witnesses
9	Finite CSP controls	3-SAT, 3-colour, pigeonhole, 8/9 Queens	Exact finite model sets; pigeonhole UNSAT; 92 and 352 Queens models
10	Bounded program induction	500 tasks; 2,000 held-out inputs	2,000/2,000 correct; opaque operator names

Table 3: Scientific headline results, part II.

7.1 Selected interpretation

The 200,000-hop result is a narrow frontier-width-one composition test: query time rose from 0.0032 s at 1,000 hops to 0.6449 s at 200,000. It does not imply the same scaling for heavily branching graphs.

The million-distractor test kept a fixed 128-hop useful path and added unrelated indexed records. Query time stayed near half a millisecond because those records used unrelated subject/relation keys and were not traversed; building the million-event Ledger raised process peak memory to about 1.0 GB.

In the alternative-state test, 40,000 small random cases were checked by independent exact enumeration across MAX_MIN, COUNT, MIN_PLUS and BOOLEAN. A separate 1,000-choice stress repeatedly split two ways and reconverged immediately; obsolete choice conditions collapsed, so the final result did not carry a 2^{1000} history cross-product.

The CSP tests demonstrate finite relational expressiveness, not SMT-solver equivalence. THREADS has no CDCL/DPLL(T), arithmetic theory solver, bit-vector theory, or clause learning.

8 Ten applied AI and enterprise experiments

The same engine was also tested on ten structured use patterns. Each had an initial 100-case benchmark and a separate randomized robustness generator.

#	Use case	Initial	Randomized	Property tested
11	Correctable AI/customer memory	400/400	900/900	current/history, correction, retraction, re-learning
12	Access control / entitlement revocation	300/300	900/900	authorization chains and immediate revocation
13	Compliance conflict / abstention	100/100	300/300	answer, contradiction, convergence, unresolved alternatives
14	Workflow dependency invalidation	300/300	600/600	selective invalidation of dependent work
15	RAG / answer provenance	400/400	600/600	corrected evidence and current/historical witness

Table 4: Applied benchmark results, part I.

#	Use case	Initial	Randomized	Property tested
16	Incident dependency / resilience	300/300	900/900	redundant paths and removal of all support
17	Data lineage / source correction	200/200	600/600	current and historical source trace
18	Multi-tenant / environment isolation	400/400	1329/1329	context isolation under identical entity IDs
19	Supply-chain recall tracing	200/200	475/475	multi-hop recall, safe abstention, withdrawal/history
20	Agent plan / state recovery	500/500	844/844	exact resume state after plan/step updates
Combined		3,100/3,100	7,448/7,448	10,548/10,548 checks

Table 5: Applied benchmark results, part II.

These tests are synthetic demonstrations of state/reasoning patterns, not production deployments. The most direct possible uses suggested by the results are a correctable AI memory/provenance layer, an agent state store, and an exact enterprise dependency/policy substrate when inputs are already structured.

9 Five diagnostic microtests

A frozen 60-category suite was first run against the earlier implementation. It passed 5,230/5,830 examples across 54/60 fully passing categories. After structural fixes, the same suite passed 5,830/5,830. Five tests that exposed useful weaknesses are:

#	Category	Earlier	Current	What was corrected
41	Explicit negation	0/100	100/100	multi-hop terminal negative evidence was no longer collapsed into UNKNOWN
43	Irrelevant contradiction branch	0/100	100/100	contradiction stopped acting as a permanent global flag
46	Ambiguous branches converge	0/100	100/100	alternatives were resolved from final outcomes rather than an early ambiguity flag
49	All ambiguous branches die	0/100	100/100	identical empty outcomes now resolve to UNKNOWN
55	Proof witness recovery	0/100	100/100	a world-consistent witness is exposed without discarding the full proof DAG

Table 6: Five small tests retained as regression checks.

The benchmark answers were not stored in the Engine. The test suite stayed fixed while the code changed. A later independent possible-outcome fuzz test also matched 20,000/20,000 cases across all four algebras.

10 What the experiments currently support

The current evidence supports these limited statements for the tested snapshot:

- exact finite propagation over the four declared algebras on the tested distributions;
- current and historical temporal queries with retraction, functional supersession, context isolation and provenance;
- distinct UNKNOWN, NEGATED, CONTRADICTED and unresolved alternative outcomes;
- exact reconvergence of alternatives when their remaining consequences become identical;
- exact finite natural joins and bounded operator-program search on the tested controls;
- plausible structured uses in memory, provenance, dependency, authorization, lineage and agent state.

11 What the experiments do not support

The tests do not establish that THREADS is better than a Transformer, LLM, database, SAT/SMT solver, or production policy system. They do not test natural-language understanding, semantic retrieval, perception, common-sense knowledge, or open-ended generation. Relation programs are normally supplied rather than inferred from prose. Program search is bounded by a supplied finite operator library and depth. Exact alternatives can still grow combinatorially when many choices remain relevant and fail to reconverge. Event time is a Python floating-point field in the tested snapshot, so applications requiring exact real-valued simultaneity would need a stronger time representation or policy. The applied benchmarks are synthetic and do not substitute for real-domain baselines, concurrency/security testing, persistence testing, or independent reproduction.

12 Minimal reference use and reproduction

A minimal current/historical query is:

```
from srmh.engine import Engine
from srmh.ledger import Event
from srmh.algebra import BOOLEAN

engine = Engine(functional={"preference"})
engine.ledger.add(Event(
    id="e1", subject="user-1", relation="preference",
    object="email", time=10.0, source="example"))
engine.ledger.add(Event(
    id="e2", subject="user-1", relation="preference",
    object="sms", time=20.0, source="example"))

old, _, _ = engine.walk(
    "user-1", ["preference"], time=15, algebra=BOOLEAN)
now, _, meta = engine.walk(
    "user-1", ["preference"], time=25,
    algebra=BOOLEAN, proof=True)

print(old.status, old.values) # ANSWER ('email',)
print(now.status, now.values) # ANSWER ('sms',)
print(meta["minimal_witness"])
```

From the release root:

```
python benchmarks/paper10_benchmark.py --test all \
    --out results/paper10/reproduced_all.json
python benchmarks/applied10_benchmark.py
python benchmarks/applied10_robustness.py
python benchmarks/micro60_suite.py source \
    --out results/micro/micro60_reproduced.json
PYTHONPATH=source python -m unittest discover -s source/tests -v
```

The repository includes the exact tested source, benchmark generators, stored raw results, reproduction notes, and a SHA-256 manifest.

13 Publication, contact and licensing

Publication date: 6 September 2026

Contact: rickeyernest1990@gmail.com

Licensing

Licensing warning - noncommercial use. Different parts of this publication are licensed under different noncommercial licences. Use the licence that applies to the material you are reusing.

Documentation, explanatory text, figures and tables. The copyrightable documentation and presentation in this PDF are licensed under the Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0). This permits sharing and adaptation subject to the licence terms, including attribution and the noncommercial-use restriction.

Source code and code listings. Source code implementing THREADS, including implementation code and source-code listings reproduced in this document, is licensed separately under the PolyForm Noncommercial License 1.0.0.

Commercial use. Neither of the licences above grants unrestricted commercial-use rights. For commercial licensing enquiries, contact rickeyernest1990@gmail.com.

Scope note. These notices apply to the copyrightable expression, documentation, figures, presentation, and software supplied with this work to the extent protected by applicable law. They are not intended to restrict mathematical facts, formulas, abstract ideas, or other material that is not subject to copyright protection. The notices do not themselves establish or imply any claim about patentability, novelty, or ownership of ideas that may be governed by separate law or prior art.

14 Conclusion

THREADS is the part I kept from a larger transformerless-AI attempt that failed as a complete system. The temporal Ledger and relational engine were worth separating because they could be tested independently with exact graders.

On the current synthetic benchmarks, that subsystem is behaving consistently across long chains, temporal revision, shuffled histories, indexed distractors, explicit negation and contradiction, reconverging alternatives, finite constraints, provenance and bounded program search. The appropriate next step is independent reproduction and matched external baselines, not a claim that the larger original AI goal has been solved.